Java Assignment

1. What is OOPs?

Object Oriented Programming (OOPs) is a programming concept that organizes software design around **objects instead of functions and logic**. In OOPs, programs are divided into small reusable pieces called **objects** which interact with each other.

An **object** represents a real-world entity such as a car, student, bank account, etc. Each object contains **data (variables)** and **methods (functions)**.

OOPs helps programmers build programs that are **easy to understand, maintain, and reuse**.

Main Concepts of OOPs

Encapsulation
Encapsulation means wrapping data and methods together inside a class and restricting direct access to some data.

Inheritance
Inheritance allows one class to inherit the properties and methods of another class.

Polymorphism
Polymorphism means performing a single action in different ways.

Abstraction
Abstraction hides unnecessary details and shows only essential features.

Example

```
class Student {
    String name;
    int age;

    void display() {
        System.out.println(name + " " + age);
    }
}
```

Here **Student** is a class and name and age are properties of the object.

---

2. Java Bytecode and JVM

Java programs do not run directly on a computer. They are first converted into **bytecode** and then executed by the **Java Virtual Machine (JVM)**.

Java Bytecode

Bytecode is an **intermediate code generated by the Java compiler** after compiling a Java program.

When we compile a Java program:

`Hello.java → Hello.class`

The **.class file contains bytecode**.

Bytecode is **platform independent**, meaning it can run on any system that has JVM installed.

JVM (Java Virtual Machine)

JVM is a virtual machine that **runs Java bytecode**.

Functions of JVM:

- Converts bytecode into machine code
- Manages memory
- Provides security
- Makes Java platform independent

Because of JVM, Java follows the principle:

Write Once, Run Anywhere (WORA).

3. Primitive Data Types in Java

Primitive data types are the **basic data types available in Java** used to store simple values.

Java has **8 primitive data types**.

1. byte

Used to store small integer values.

Size: 1 byte

Example:

```
byte a = 10;
```

2. short

Stores slightly larger integers.

Size: 2 bytes

Example:

```
short b = 200;
```

3. int

Most commonly used integer type.

Size: 4 bytes

Example:

```
int number = 5000;
```

4. long

Stores very large integer values.

Size: 8 bytes

Example:

```
long population = 1000000;
```

5. float

Used to store decimal numbers.

Size: 4 bytes

Example:

```
float price = 10.5f;
```

6. double

Stores larger decimal values.

Size: 8 bytes

Example:

```
double pi = 3.14159;
```

7. char

Stores a single character.

Size: 2 bytes

Example:

```
char grade = 'A';
```

8. boolean

Stores true or false values.

Example:

```
boolean status = true;
```

4. Difference Between 10.3 and 10.3F

In Java, decimal numbers are treated as **double by default**.

So:

```
10.3
```

is treated as a **double value**.

But when we write:

```
10.3F
```

the **F indicates float**.

Example

```
double a = 10.3;
float b = 10.3F;
```

Differences

| Value | Type | | --- | --- | | 10.3 | Double | | 10.3F | Float |

Float uses **less memory** compared to double.

---

5. Operators in Java

Operators are symbols used to perform **operations on variables and values**.

Types of Operators in Java

1. Arithmetic Operators

Used for mathematical operations.

Operators:

```
+  -  *  /  %
```

Example:

```
int a = 10;
int b = 5;

System.out.println(a + b);
System.out.println(a - b);
System.out.println(a * b);
System.out.println(a / b);
```

---

2. Relational Operators

Used to compare two values.

Operators:

```
==  !=  >  <  >=  <=
```

Example:

```
int a = 10;
int b = 20;

System.out.println(a > b);
```

3. Logical Operators

Used to combine conditions.

Operators:

```
&&  ||  !
```

Example:

```
int age = 20;

if(age > 18 && age < 60) {
    System.out.println("Eligible");
}
```

---

4. Assignment Operators

Used to assign values.

```
=  +=  -=  *=  /=
```

Example:

```
int a = 10;
a += 5;
```

---

5. Unary Operators

Operate on a single variable.

```
++  --
```

Example:

```
int a = 5;
a++;
```

---

6. Type Conversion in Java

Type conversion means **changing one data type into another data type**.

Types of Type Conversion

1. Implicit Conversion (Widening)

Automatic conversion done by Java.

Example:

```
int a = 10;
double b = a;
```

Here int is automatically converted to double.

---

2. Explicit Conversion (Casting)

Done manually by the programmer.

Example:

```
double a = 10.5;
int b = (int)a;
```

Here double is converted to int.

7. Applications and Benefits of OOPs

OOPs is widely used in modern software development.

Applications

- Software development
- Game development
- Web applications
- Mobile apps
- Banking systems
- Real-time systems

Benefits

Code Reusability
Classes can be reused in different programs.

Security
Encapsulation protects data.

Easy Maintenance
Programs are easier to update and modify.

Flexibility
Polymorphism allows flexibility in program design.

Modularity
Large programs can be divided into small modules.

---

8. Tokens in Java

Tokens are the **smallest elements of a Java program**.

Java tokens are divided into five categories.

Types of Tokens

Keywords

Reserved words that have special meaning.

Examples:

```
class
int
public
static
void
```

Identifiers

Names given to variables, classes, and methods.

Example:

```
studentName
age
calculate()
```

Literals

Constant values used in the program.

Examples:

```
10
3.14
'A'
true
```

Operators

Symbols used to perform operations.

Examples:

```
+
-
*
/
```

Separators

Used to separate program elements.

Examples:

```
( ) { } [ ] ; , .
```

---

9. Five Vector Methods in Java

Vector is a **dynamic array class** in Java.

Example

```
import java.util.Vector;
```

1. add()

Adds an element to the vector.

```
Vector<String> v = new Vector<>();

v.add("Apple");
v.add("Banana");
```

---

2. get()

Returns element at specific index.

```
System.out.println(v.get(0));
```

---

3. remove()

Removes an element.

```
v.remove("Apple");
```

---

4. size()

Returns number of elements.

```
System.out.println(v.size());
```

---

5. contains()

Checks if element exists.

```
System.out.println(v.contains("Banana"));
```

---

10. Constructor in Java

A constructor is a **special method used to initialize objects**.

It has the **same name as the class** and **no return type**.

Example

```
class Student {

    String name;

    Student() {
        name = "Ashwith";
    }

    void display() {
        System.out.println(name);
    }

    public static void main(String args[]) {
        Student s = new Student();
        s.display();
    }
}
```

When the object is created, the constructor runs automatically.

---

11. Recursion

Recursion is a process where a **method calls itself** repeatedly until a condition is met.

It is commonly used in **factorial, searching, and tree problems**.

Example: Factorial

```
class Factorial {

    int fact(int n) {

        if(n == 1)
            return 1;

        else
            return n * fact(n-1);
    }

    public static void main(String args[]) {

        Factorial f = new Factorial();

        System.out.println(f.fact(5));
    }
}
```

Output:

```
120
```

---

12. Class in Java

A class is a **blueprint used to create objects**.

It contains **variables and methods**.

Example

```
class Car {

    String brand;
    int speed;

    void display() {
        System.out.println(brand + " " + speed);
    }

}
```

Creating Object

```
public class Main {

    public static void main(String args[]) {

        Car c = new Car();

        c.brand = "BMW";
        c.speed = 120;

        c.display();
    }
}
```

---

13. Scope of Variables in Java

Scope defines **where a variable can be accessed in a program**.

Types of Variable Scope

1. Local Variables

Declared inside a method.

```
void display() {
    int a = 10;
}
```

Accessible only within that method.

---

2. Instance Variables

Declared inside a class but outside methods.

```
class Test {
    int number;
}
```

Each object has its own copy.

---

3. Static Variables

Declared with the **static keyword**.

```
class Test {
    static int count = 0;
}
```

Shared by all objects of the class.

Conclusion

Java is a powerful object-oriented programming language that supports modular, reusable, and secure programming. Concepts such as OOPs, data types, operators, constructors, recursion, and classes help developers create efficient and scalable applications. Understanding these basic Java concepts is essential for building strong programming skills and developing real-world software applications.